mbusgw-t2r

User Guide

Document Number: DO-T2RUG-EN-01A
Publication Date: November 2026
Revision Update Information: This is a new manual.
Operating System: Linux (glibc), kernel 4.x or later
Software Version: mbusgw-t2r X.00-13ECO09

NaPi World & StarLet Squad collaboration

mbusgw-t2r User Guide

Copyright © 2026 NaPi World & StarLet Squad collaboration

Legal Notice

The information contained herein is subject to change without notice. NaPi World & StarLet Squad collaboration shall not be liable for technical or editorial errors or omissions contained herein.

The software described in this manual is licensed under the Apache License, Version 2.0; see the LICENSE and NOTICE files of the distribution.

The software described in this manual is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

MODBUS is a registered trademark of Schneider Electric USA, Inc. Linux is a registered trademark of Linus Torvalds. UNIX is a registered trademark of The Open Group.

Contents

Preface iv

1. About This Manual iv
2. Intended Audience iv
3. Document Structure iv
4. Related Documents iv
5. Conventions iv

Chapter 1. What the Gateway Does 1

Chapter 2. Before You Start 2

Chapter 3. The Settings File 3

3.1. The serials Section 3
3.2. The listeners Section 4

Chapter 4. Running the Gateway 5

4.1. The First Test with mbpoll 5
4.2. On OpenWrt 6

Chapter 5. The Time Stamp Pseudo Device 7

Chapter 6. Reading the Log 8

Chapter 7. Troubleshooting 9

7.1. The Symptom, the Cause, the Action 9
7.2. The Message Codes Reference 10
7.3. If Nothing Helps 13

Preface

1. About This Manual

This manual describes the configuration, the operation and the troubleshooting of mbusgw-t2r --- a gateway which translates MODBUS TCP requests coming from the network into MODBUS RTU transactions on a serial (RS-485 or RS-232) line, and returns the answers back.

2. Intended Audience

This manual is written for a reader with no prior MODBUS or Linux administration experience: every step is spelled out and nothing is assumed. An experienced engineer may go straight to Chapter 3 (the settings file) and Chapter 7 (the troubleshooting).

3. Document Structure

Chapter 1 explains what the gateway does. Chapter 2 is the pre-flight checklist. Chapter 3 describes the settings file key by key. Chapter 4 covers the start, the command line options and the first end-to-end test. Chapter 5 describes the built-in Time Stamp pseudo device. Chapter 6 teaches how to read the log. Chapter 7 is the troubleshooting reference: the symptom, the cause, the action, and the reference of all the message codes.

4. Related Documents

README.md --- the installation sequence (the StarLet package, the build, `make install`) and the automatic build in GitLab CI (`.gitlab-ci.yml`). MODBUS over Serial Line Specification and Implementation Guide and MODBUS Application Protocol Specification --- the protocol itself, available from modbus.org.

5. Conventions

Convention	Meaning
`Monospace type`	Code examples, file names, commands and interactive screen displays.
`%T2R-E-CODE`	A message code as it appears in the gateway log; see Section 7.2.
`<...>`	A placeholder to be replaced by an actual value, e.g. a device name.
`$`	The shell prompt of an unprivileged user; do not type it.
`Ctrl/C`	Hold down the key labeled Ctrl while you press the key C.

Chapter 1. What the Gateway Does

You have a device (an electricity meter, a temperature controller, a PLC ...) which talks MODBUS RTU over a serial line. And you have a program (a SCADA, a script, `mbpoll` ...) which talks MODBUS TCP over the network. The gateway sits between them and translates, as Figure 1 shows.

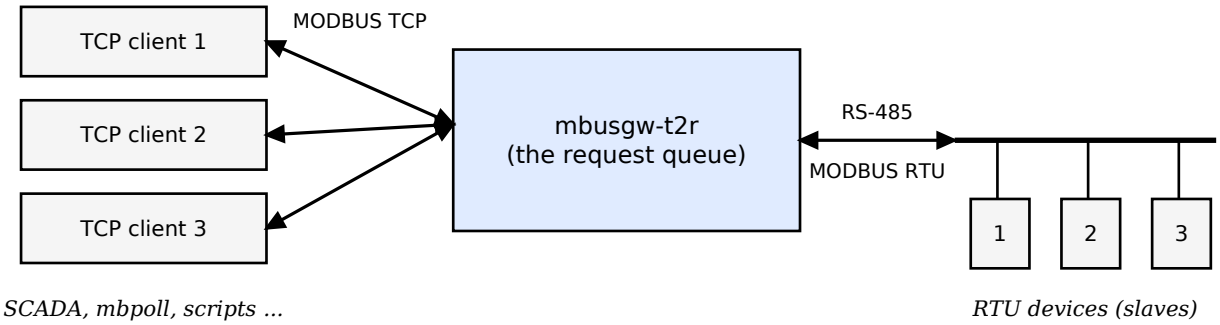


Figure 1. The data path of the gateway

Several TCP clients may connect at the same time: the gateway queues them so that only one request at a time goes to the serial line --- an RTU line physically cannot carry two transactions at once.

Several devices may share one RS-485 line. They are told apart by the slave address inside the MODBUS request itself; the gateway does not need to know their addresses at all.

Chapter 2. Before You Start

Walk this checklist once --- it saves an hour of guessing later.

1. The gateway is installed (see README.md). After the installation you have the program `/opt/mbusgw-t2r/sbin/mbusgw-t2r` and the settings file `/opt/mbusgw-t2r/etc/modbus-t2r_settings.conf`.

2. You know which serial port the device is connected to. On Linux it is a file like `/dev/ttyS0` or `/dev/ttyUSB0`. If unsure, plug the USB adapter out and in, then run:

```
$ dmesg | tail
```

3. You know the line parameters of the device: the speed, the data bits, the parity and the stop bits. They are in the device manual; the most common set is `9600, 8, N, 1`.

4. Your user may open the port. Check the group of the port file:

```
$ ls -l /dev/ttyUSB0
crw-rw---- 1 root dialout 188, 0 ... /dev/ttyUSB0
```

If the group is `dialout`, add yourself to it and re-login (or simply run the gateway with `sudo`):

```
$ sudo usermod -a -G dialout $USER
```

Chapter 3. The Settings File

The settings file has two sections: `serials` (the serial lines) and `listeners` (the TCP ports). A minimal working example:

```
serials = (
{ device = "/dev/ttyUSB0";
  chars  = "9600, 8, N, 1";
  iotmo  = 1000;
}
);

listeners = (
{ bind   = "TCP:0.0.0.0:502";
  target = "/dev/ttyUSB0";
  connlm = 8;
}
);
```

Read it as: "open the port /dev/ttyUSB0 at 9600-8-N-1 and wait for an answer up to 1000 ms; listen for TCP clients on every interface, port 502, and forward them to that port".

3.1. The serials Section

One record per serial port. Table 3-1 lists every key.

Key	Required	Meaning and the allowed values
`device`	yes	The serial port file, e.g. /dev/ttyUSB0.
`chars`	yes	The line parameters: speed, data bits, parity, stop bits. Speed 50..4000000; data bits 5..8; parity N (none), E (even), O (odd); stop bits 1..2.
`iotmo`	no	How long to wait for the answer of the device, milliseconds. Default 1000. Take it with a margin over the slowest device on the line: an answer which comes later is lost.
`lategd`	no	After a failed request (no answer, a broken answer) listen to the line for this many milliseconds more and throw away what comes (LATEANS): a late answer must not go to the next request. Costs time only after a failure: a request to a dead device takes iotmo + lategd. 0..60000, default iotmo / 4; 0 --- only the 3.5 character pause, as before X.00-13ECO08.
`rs485`	no	1 --- ask the kernel to drive the RS-485 direction control (only for the ports which support it). Default 0.
`ts_enabled`	no	1 --- enable the Time Stamp pseudo device (Chapter 5). Default 0.
`ts_unit`	no	The slave address the pseudo device answers on. Default 135.
`ts_fncode`	no	The function code it answers to. Default 4.
`ts_base_reg0`	no	The first register of the time stamp. Default 135.

If a record is wrong, the gateway skips it and says why --- with the allowed range right in the message:

```
%T2R-E-SERRANGE, [serial #00:</dev/ttyUSB0>] --- speed (baud) 31 is out of range [50..4000000]
```

USB adapters: unplugging is fine. Since X.00-13ECO09 the gateway notices that the adapter has gone (DEVLOST) --- also when nobody is polling --- and closes the port at once. It tries to open the port again by its name every second; meanwhile the clients get

the exception 10 (GATEWAY_PATH_UNAVAILABLE), not 4. When the adapter is back, DEVREADY is printed and the work goes on, no restart is needed. An adapter absent at the start does not stop its TCP port (DEVABSENT).

Use a permanent name. The kernel numbers ttyUSB0, ttyUSB1, ... in the order the adapters appear, so after a replug the same adapter may get another number. Put a permanent name into device (and into target of the listener): the name by the USB socket from ls -l /dev/serial/by-path/ (the ports of a 4-port FT4232H differ by the :1.0 ... :1.3 part), or your own name made by a udev rule:

```
SUBSYSTEM=="tty", ENV{ID_PATH}=="platform-xhci-hcd.0-usb-0:1.1:1.0", SYMLINK+="rs485-1"
```

in /etc/udev/rules.d/99-rs485.rules gives /dev/rs485-1 (see udevadm info /dev/ttyUSB0 for the values).

3.2. The listeners Section

Key	Required	Meaning and the allowed values
`bind`	yes	Where to listen: TCP:<IP address>:<port>. The port is 1..65535; 0.0.0.0 means every interface. UDP is not supported.
`target`	yes	Which serial port to forward to. Must match a device from serials exactly, character by character.
`connlm`	no	How many TCP clients may be connected to this port at the same time; one more is disconnected at once (CONNLIMIT). Default 32, range 1..512.
`idletmo`	no	A client which sends nothing for this many seconds is disconnected (IDLETMO). Default 60, range 1..86400.
`iotmo`	no	Reserved, not used yet.

Port 502 is the standard MODBUS TCP port, but on Linux the ports below 1024 need the root privileges. If you do not want to run as root, take e.g. `5502` and point your client there. Several listeners may share one target: the gateway serializes their requests.

Chapter 4. Running the Gateway

By hand, in a terminal (the messages go straight to the screen):

```
$ /opt/mbusgw-t2r/sbin/mbusgw-t2r /settings=/opt/mbusgw-t2r/etc/modbus-t2r_settings.conf
```

Option	Meaning
`/settings=<file>`	The settings file (Chapter 3).
`/trace`	The verbose tracing: every request and answer is dumped in hex. Priceless at the first run.
`/logfile=<file>`	Write the log to a file instead of the screen.
`/logsize=<octets>`	Rotate the log file above this size.

A healthy start prints (shortened):

```
%T2R-I-REVISNF, Rev: T2R X.00-06/aarch64(built at ...) (REV: 00.06.00)
%T2R-I-DEVADDED, Added device #00 [</dev/ttyUSB0>, Chars: <9600, 8, N, 1>, ...] --- added
%T2R-I-LSNADDED, Added listener #00 [Target: </dev/ttyUSB0>, Net: <TCP:0.0.0.0:502>, ...] --- added
%T2R-S-DEVREADY, Device </dev/ttyUSB0> [9600 baud, answer tmo: 1000 msec, t3.5: 4010 usec] --- is r
%T2R-S-LSNRRDY, [#3] Listener 0.0.0.0:502 [Target: </dev/ttyUSB0>] --- is ready
```

Two lines matter most: DEVREADY (the port is open) and LSNRRDY (the TCP port is listening). If you see both, the gateway is up. Figure 2 shows what happens to every request after that.

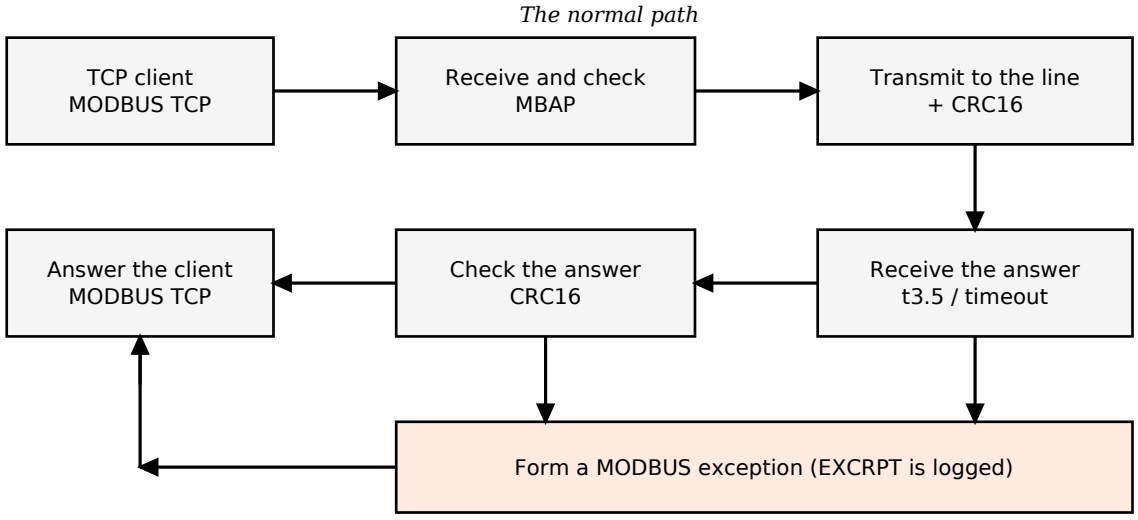


Figure 2. The lifecycle of one request

To stop the gateway press Ctrl/C once and wait a second. To toggle the tracing of a running gateway without a restart:

```
$ kill -USR1 <pid>
```

4.1. The First Test with mbpoll

`mbpoll` is a small free MODBUS client (`sudo apt install mbpoll`). Read 3 holding registers from the device with the slave address 1:

```
$ mbpoll -a 1 -r 1 -c 3 -1 127.0.0.1 -p 502
```

If the numbers come back --- everything works, end to end.

4.2. On OpenWrt

On an OpenWrt router install the package for your board from the Releases page (the architecture is the `DISTRIB_ARCH` line of `/etc/openwrt_release`): `.ipk` for OpenWrt 24.10, `.apk` for OpenWrt 25.12. The package needs nothing else, the router needs no network:

```
# opkg install ./mbusgw-t2r_0.13.9-*_mipsel_24kc.ipk
# apk add --allow-untrusted ./mbusgw-t2r_0.13.9-*_mipsel_24kc.apk
```

The files are at other places than on Debian: the program is `/usr/sbin/mbusgw-t2r`, the settings file is `/etc/mbusgw-t2r/modbus-t2r_settings.conf`. The service is switched off after the install. Tune the settings file, then switch the service on and start it:

```
# uci set mbusgw-t2r.main.enabled=1 && uci commit mbusgw-t2r
# /etc/init.d/mbusgw-t2r start
# logread -e mbusgw-t2r
```

The last command shows the messages of the gateway (they go to the system log). The USB serial adapter needs its kernel module: `kmod-usb-serial-ftdi`, `kmod-usb-serial-ch341` or `kmod-usb-serial-cp210x`, by the chip of the adapter.

Chapter 5. The Time Stamp Pseudo Device

The gateway can answer by itself --- without touching the line --- to a read request sent to a special slave address. The answer is the current time of the gateway host. This lets a SCADA read the precise time over the plain MODBUS.

With `ts_enabled = 1; ts_unit = 135; ts_fncode = 4; ts_base_reg0 = 135;` a request "read N input registers starting at 135 from the slave 135" returns the registers of Figure 3.

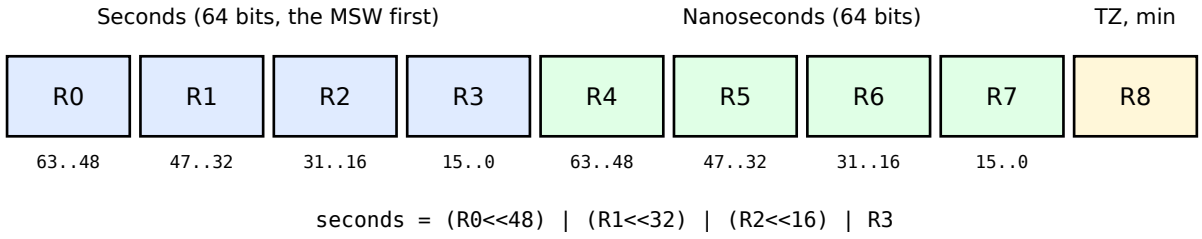


Figure 3. The Time Stamp registers layout

Registers requested	You get
4	The seconds of the UNIX epoch, 64 bits, the most significant 16-bit word first.
8	Plus the nanoseconds, packed the same way into R4..R7.
9	Plus R8 = the local timezone offset, in signed minutes (e.g. Moscow = +180).

Any other register count in a TS request is answered with the MODBUS exception ILLEGAL_DATA_ADDRESS --- this is on purpose.

Chapter 6. Reading the Log

Every important event is one line with a code:

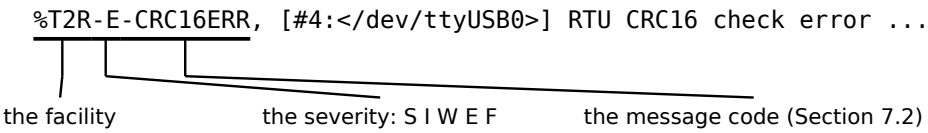


Figure 4. The anatomy of a log message

The severity letter is: S --- success, I --- information, W --- warning, E --- error, F --- fatal. The part `[#4:</dev/ttyUSB0>]` means: the file descriptor #4, this serial port.

Grep the log by the code --- this is exactly why the codes exist:

```
$ grep CRC16ERR /var/log/mbusgw-t2r.log
```

Chapter 7. Troubleshooting

The golden rule: run with /trace and read the log. The gateway always says what it dislikes.

7.1. The Symptom, the Cause, the Action

You see	It means / what to do
`%T2R-E-DEVOPNERR, ... errno: 2`	The port file does not exist: the adapter is unplugged or the name is wrong. Run dmesg | tail after plugging; fix device in the settings. Since X.00-13ECO09 the TCP port is opened anyway (DEVABSENT), the device is tried every second.
`%T2R-E-DEVLOST, ...`	The USB adapter has gone (unplugged, a hub reset, a power dip); the clients get the exception 10. Nothing to do if it comes back: DEVREADY follows. If not: dmesg | tail, the cable, the power; if it came back under another ttyUSB number, use a permanent name (3.1).
`%T2R-W-DEVABSENT, ...`	At the start the serial port could not be opened (the line above says why); its TCP port works and answers the exception 10. Plug in the adapter, or fix device.
`The client gets the exception 0x0A (10)`	The serial port of this TCP port is absent or lost: see DEVLOST, DEVABSENT.
`%T2R-E-DEVOPNERR, ... errno: 13`	Permission denied. Run with sudo, or add yourself to the dialout group.
`%T2R-E-DEVOPNERR, ... errno: 16`	The port is busy. Find the holder: sudo fuser /dev/ttyUSB0, and stop it.
`%T2R-E-NOANSWER, ...`	The request went out, nothing came back: 1) a wrong slave address in the client request; 2) the wiring (swap A/B of RS-485); 3) a wrong speed/parity --- re-check chars; 4) the device is off.
`%T2R-E-CRC16ERR, ...`	The octets arrive damaged: 1) chars does not match the device (the most common!); 2) the line noise --- the grounding, the termination, the cable length; 3) two masters on one line.
`CRC16ERR on a part of the requests after a reboot or a replug`	A gateway older than X.00-13ECO07 kept the CR-to-NL translation (icrnl) of a fresh port: an octet 0x0D became 0x0A. Update; for an old gateway: ExecStartPre=/bin/stty -F /dev/ttyUSB0 raw (one per port) in the unit.
`%T2R-E-BADFRAME, ...`	Something arrived, but it is not a sane MODBUS frame: usually the same causes as CRC16ERR; 400+ octets of garbage mean a foreign, non-MODBUS device on the line.
`CRC16ERR / BADFRAME now and then, on an FTDI USB adapter only`	The adapter gives the answer by pieces, once in 16 msec (latency_timer). Since X.00-10 the gateway waits for the whole answer of the functions 01-06, 15, 16, 23; the others are still cut. Set latency_timer to 1 msec (the answers get faster too): copy /opt/mbusgw-t2r/share/doc/mbusgw-t2r/99-ftdi-latency.rules to /etc/udev/rules.d/, run sudo udevadm control --reload && sudo udevadm trigger --subsystem-match=usb-serial; cat /sys/bus/usb-serial/devices/ttyUSB0/latency_timer must print 1.
`%T2R-W-FRAMETMO, ...`	The answer started but never finished: a very slow or hanging device. Raise iotmo; check the cable.

You see	It means / what to do
`%T2R-W-LATEANS, ... after NOANSWER`	The device answered, but later than iotmo; the answer has been thrown away. Raise iotmo with a margin over the slowest device on the line.
`%T2R-W-ECHOMISM, ...`	The answer to a write (functions 5, 6, 15, 16) is not the echo of this request: usually a late answer to an earlier one (raise iotmo); with a fast device --- two masters on one line; on every write to one device --- the device does not echo as the specification wants: the write is done but reported as failed, tell us the model.
`%T2R-W-EXCRPT, ... (SERVER_DEVICE_FAILURE)`	The gateway told the client "the serial side failed". The real reason is in the previous log line (NOANSWER, CRC16ERR ...).
`%T2R-W-CONNLIMIT, ...`	The port already has connlm clients. If they are all real, raise connlm of this listener; otherwise find who keeps the connections: ss -tn.
`%T2R-W-IDLETMO, ...`	A client has sent nothing for idletmo seconds. Normal for a client which polls rarely --- it reconnects; to keep the connection, raise idletmo.
`%T2R-E-TXDRAINTMO, ...`	The USB adapter took the request but did not send it out; the request gets exception 4. The adapter or its USB link hangs: dmesg | tail; replug the adapter or reboot the board. Since X.00-13ECO01 the line is not blocked by it.
`DEVLOCKERR / DEVBUSY on every request of one port`	Another request holds the line and does not end. Update to X.00-13ECO01 or newer (an older gateway can hang forever in the serial driver). Before the restart save what the threads do: for t in /proc/$(pidof mbusgw-t2r)/task/*; do echo $t $(cat $t/wchan); done
`%T2R-E-SESSFAIL, ... pthread_create()->11`	Too many TCP clients at once: every client takes a thread, and systemd limits the service (TasksMax, 578 on a small board). Find who keeps so many connections (ss -tn); if they are all real, set TasksMax=2048 in the [Service] section of the unit and run systemctl daemon-reload.
`%T2R-E-LSNRERR, ... errno: 98`	The TCP port is taken: a second gateway instance, or another program. Find it: sudo ss -tlnp | grep 502.
`%T2R-E-LSNRERR, ... errno: 13`	The ports below 1024 need root. Run with sudo, or take a port above 1024 (e.g. 5502).
`%T2R-E-SERRANGE, ... out of range [a..b]`	A settings value is out of its range; the allowed range is right in the message. Fix the value.
`%T2R-E-NOSERIALS, No serials have been defined!`	Not a single serials record survived the validation. Read the error lines above --- each rejected record says why.
`The data is wrong or shifted`	The byte order confusion on the client side: the MODBUS registers are the big-endian 16-bit words; check how the client assembles the 32/64-bit values.

7.2. The Message Codes Reference

Code	Sev	When it appears
`REVISNF`	I	At the start: the program version. Quote it when asking for help.
`DEVREADY`	S	The serial port is open; the actual baud, the answer timeout and the t3.5 interval are shown.
`LSNRRDY`	S	The TCP port is listening; the address, the port and the target device are shown.

Code	Sev	When it appears
NETCONN	S	A TCP client has connected; its address:port and the listener are shown.
NETDISCN	S	A TCP client has disconnected; its address:port are shown.
DEVOPNERR	E	The serial port cannot be opened; errno says why (2 = no such file, 13 = the permissions, 16 = busy).
LSNRERR	E	bind()/listen() failed for a TCP port; errno says why (98 = taken, 13 = the privileges).
NOANSWER	E	The device did not answer within the iotmo timeout.
FRAMETMO	W	The answer began but was not completed within the timeout.
BADFRAME	E	The received frame length is outside the sane [5..256] octets range.
CRC16ERR	E	The checksum of the received frame does not match; both values are shown.
EXCRPT	W	A MODBUS exception is returned to the TCP client; the code and its name are shown.
EXITST	I	The gateway exits; the exit flag and the final status are shown.
SYSERR	E	A system call failed; its name, the return value and errno are shown. Quote the line when asking for help.
NETSYSERR	E	A system call on a TCP connection failed (recv, send, poll, accept); the connection is closed.
NETSYSWRN	W	A socket option cannot be set; the gateway keeps working.
TTYSYSERR	E	A system call on the serial port failed (poll, read); the request gets exception 4.
TTYSYSWRN	W	A minor serial port call failed (tcflush, poll/read while flushing); the gateway keeps working.
DEVSYSERR	E	A serial port setup call failed (tcgetattr, tcsetattr, cfsetspeed, mutex); the port is not used.
DEVSYSWRN	W	Waiting for the serial line took too long (pthread_mutex_timedlock); the gateway tries again.
PEERCLOSE	I	The TCP client has closed the connection — the normal end of a session.
ALLSENT	W	Nothing left to send on a TCP connection — harmless.
BADPROTO	E	A TCP request has a non-zero MBAP protocol id: the client does not speak MODBUS TCP; the connection is closed.
PDUSHORT	E	A TCP request is too short to be a MODBUS request; the connection is closed.
PDULONG	E	A TCP request is longer than 256 octets; the connection is closed.
IDLETMO	W	A TCP client has been silent for idletmo seconds (60 by default); the connection is closed.
NETRDTMO	E	A TCP client started a request but did not finish it in time; the connection is closed.
REQFAIL	W	A request is answered with a MODBUS exception; the previous lines say why.
NETPOLLEV	W	A TCP connection broke: the client reset it (e.g. the polling program was restarted); the connection is closed.
SESSFAIL	E	No thread for a new TCP client: too many connections at once (see Troubleshooting); the client is dropped.
LSNRABRT	W	A TCP listener is stopped — normal at the gateway exit.

Code	Sev	When it appears
TGTOPNERR	E	A TCP port is not opened because its serial port cannot be set up; the previous line says why. An absent device does not stop the TCP port since X.00-13ECO09 (DEVABSENT).
NOLSNRUP	E	Not a single TCP port could be opened; the gateway stops.
NOROOM	E	An answer does not fit into the buffer — an internal error, please report it.
MBAPLEN	E	The MBAP length field does not match the received data; the request is rejected.
REQSHORT	E	A request is too short to be forwarded to the serial line.
TGTNDEF	E	A listener names a target which is not in the serials section.
CFGNOSECT	E	The settings file has no serials or listeners section; the gateway stops.
CFGTRUNC	W	Too many records in a section; only the first ones are used.
SERBADREC	E	A serials record is broken or has no device; it is skipped.
SERBADOPT	E	A serials record has no chars, or it is too long; the record is skipped.
SERNORS485	W	rs485 = 1 is set, but this build has no RS485 support; the option is ignored.
SERBADCHR	E	The chars value cannot be read; write it like "9600, 8, N, 1".
SERRANGE	E	The speed, data bits or stop bits are out of range; the allowed range is in the message.
SERBADPAR	E	The parity letter is unknown; use N, E or O.
DEVADDED	I	A serials record is accepted; the line parameters are shown.
TSENABLED	I	The Time Stamp pseudo device is on for this port.
TSDISABLED	I	The Time Stamp pseudo device is off for this port (the default).
NOSERIALS	E	Not a single serials record survived the validation; the gateway stops.
LSNBADREC	E	A listeners record is broken (no bind, no target, UDP); it is skipped.
LSNBADBIND	E	The bind value cannot be read; write it like "TCP:0.0.0.0:502".
LSNRANGE	E	The TCP port number is out of range [1..65535].
LSNBADADDR	E	The IP address in bind is not a valid IPv4 address.
LSNNOTGT	E	The target of a listener is not in the serials section; the record is skipped.
LSNADDED	I	A listeners record is accepted; the address, the port and the target are shown.
NOLSNRS	E	Not a single listeners record survived the validation; the gateway stops.
CFGPARSE	E	The settings file cannot be read or has a syntax error; the file name and the line are shown.
SIGERR	E	A signal handler cannot be set — an internal error, please report it.
CFGERR	E	The configuration cannot be processed; the previous lines say why.
CFGABORT	F	The gateway stops because of the configuration errors above.
DEVLOCKERR	E	The serial line stayed busy for 39 seconds (another request holds it); the request gets exception 4.
TXNODATA	W	An empty request was not sent — harmless.
XMITERR	E	The request cannot be written to the serial port; the request gets exception 4.
XMITSHORT	E	Only a part of the request has been written to the serial port.

Code	Sev	When it appears
RXBUFFULL	W	The device sent more than 256 octets; the rest is dropped.
RXNODATA	E	The answer started but no octet was collected.
RESPMISM	W	The answer came from another slave or for another function: two masters or two devices with one address.
BADSPEED	W	The speed is not a standard one; 9600 is used.
NOTTTY	W	The device is not a serial port.
RS485OFF	W	The RS485 mode cannot be switched on for this port; the port works without it.
RS485ON	S	The RS485 mode is on for this port.
DEVINIERR	E	The serial port cannot be set up; the previous line says why.
DEVINIT	I	The serial port is set up.
DEVCLSERR	E	The serial port cannot be closed.
DEVNOTINI	W	Closing a serial port which was never opened — harmless.
DEVBUSY	E	The serial line cannot be taken for a request; the request gets exception 4.
CONNLIMIT	W	A new TCP client is disconnected at once: the port already has connlm clients.
LSNOPTRNG	E	A listeners option (connlm, idletmo) is out of range; the record is skipped.
TXDRAINTMO	E	The request was not sent out by the serial adapter in time; it is dropped, the request gets exception 4.
ECHOMISM	W	The answer to a write request is not its echo (another address or value); the request gets exception 4.
LATEANS	W	After a failed request something came from the line within lategd --- a late answer; it is dropped.
DEVLOST	E	The serial port has gone (an USB adapter unplugged); it is closed and opened again every second, the requests get exception 10 meanwhile.
DEVABSENT	W	The serial port cannot be opened at the start; the TCP port works anyway, the serial port is opened every second.

7.3. If Nothing Helps

Collect and attach to your question: 1) the full start-up log with /trace (from REVISNF to the first error); 2) your settings file; 3) the output of `ls -l <device>` and `dmesg | tail -20`; 4) the exact model of the RTU device and its documented line parameters. With these four things the problem is almost always visible at a glance.